Sécurité

1. Quel est le vrai risque dans StudyAI ?

Cette section existe parce que la réponse n'est pas celle qu'on croit, et qu'une protection qu'on ne sait pas justifier est une protection qu'on abandonne à la première difficulté.

L'énoncé du mini-projet exige : *« jamais de données personnelles brutes envoyées telles quelles au LLM »*. Appliquée à un CV, l'exigence est évidente — un CV n'est fait que de données personnelles. Appliquée à un **support de cours**, elle demande à être réexaminée.

Ce qui n'est pas le risque

Les données personnelles de l'étudiant. Un chapitre de systèmes d'exploitation ne contient ni son nom ni son numéro. L'étudiant importe son propre cours, pour son propre usage.

Les noms de personnes. Sans profil connu à l'avance ni modèle de reconnaissance d'entités nommées, un patronyme isolé n'est pas distinguable d'un terme technique. StudyAI **ne les détecte pas**, et un test le documente explicitement plutôt que de laisser croire le contraire (`test_limite_connue_le_nom_d_un_tiers_n_est_pas_detecte`).

Ce qui est le risque, et qui est démontrable

1. Les secrets oubliés dans un support. Le fichier `Seance 4/Correction (code) de l'atelier 4.py`, distribué à toute la promotion dans ce module, **contient un jeton Hugging Face en clair**. Un étudiant qui colle ce fichier comme cours expédierait une clé d'authentification vivante à un fournisseur tiers. StudyAI la masque, et un test le vérifie sur ce format précis. Ce n'est pas un scénario inventé pour la soutenance : c'est un fichier de ce module.

2. Les coordonnées de tiers sur les pages de garde. Nom de l'enseignant, adresse électronique, établissement. Ces données n'appartiennent pas à l'étudiant. **L'auteur du cours n'a pas consenti** à ce que son support parte chez un fournisseur américain. Le sujet des données, ici, c'est l'enseignant.

3. Les énoncés d'exercices. Un TD de bases de données contient volontiers une table de clients avec noms, adresses et numéros. Réels ou fictifs, rien ne permet de le savoir côté application.

> **À dire en soutenance.** Pas « on protège les données des étudiants », qui est faible et contestable. Mais : « on protège les secrets et les coordonnées de tiers qui traînent dans les supports — et voici un fichier de notre propre module qui contient une clé d'API en clair ».

---

2. Anonymisation — comment

Implémentation : `server/app/llm/redact.py`. Seize tests dédiés.

Ce qui est détecté

Type	Méthode
`SECRET`	9 familles : AWS, OpenAI, Anthropic, Hugging Face, Google, GitHub, Slack, chaînes de connexion, lignes `mot de passe:` / `token=`
`EMAIL`	motif
`TEL`	groupes de 2 à 4 chiffres, avec ou sans indicatif
`URL`	liens, profils LinkedIn et GitHub
`ADRESSE`	rue / avenue / boulevard…
`DATE_NAISSANCE`	jj/mm/aaaa
`ETABLISSEMENT`	liste : ESPRIT, INSAT, ENIT, ENSI, ISET, FST, Université
`NOM`, `ADRESSE`	uniquement si un profil est connu — **jamais le cas dans StudyAI**

Les secrets sont traités **en premier** : une chaîne de connexion contient une adresse et un mot de passe, il faut la neutraliser avant que d'autres motifs ne la découpent en morceaux.

Garde-fous contre les faux positifs

Masquer trop détruit de l'information pédagogique. Deux protections :

- une suite de chiffres de moins de huit chiffres n'est pas un numéro ;
- une suite composée uniquement d'années (`2022-2026`, `2024 2025 2026`) n'est pas un numéro — c'est une période de formation, utile au résumé.

Deux défauts corrigés, à citer

12 octobre — le motif des téléphones n'acceptait que des groupes de **deux** chiffres : `+216 55 123 456` partait **en clair**. Le défaut ne s'était jamais manifesté sur SkillMatch parce que le numéro y était connu d'avance par le profil du candidat. Dans StudyAI il n'y a aucun profil : le motif générique est la seule protection. Trouvé par un test de bout en bout, pas par relecture.

12 octobre — la vue de transparence listait les types masqués en cherchant les jetons dans la charge envoyée. Or celle-ci contient le prompt système, qui cite `[NOM_1]`, `[EMAIL_1]`, `[TEL_1]`, `[ETABLISSEMENT_1]` à titre d'exemple. L'écran affichait donc ces quatre types sur **n'importe quel** cours. Une preuve qui affirme toujours la même chose ne prouve rien. Les types viennent désormais de la table des jetons réelle.

Honnêteté méthodologique

Il s'agit d'une **pseudonymisation**, pas d'une anonymisation au sens du RGPD. Un faisceau d'informations résiduelles pourrait permettre une réidentification. Le rapport doit le dire, pas le masquer.

Garantie structurelle

Le masquage n'est pas une discipline, c'est une propriété de l'architecture :

- `app/llm/pipeline.py` expose trois fonctions, qui prennent le texte brut et anonymisent systématiquement ;
- `app/llm/client.py`, qui appelle réellement le modèle, **n'est importé nulle part ailleurs**.

Il n'existe donc aucun chemin permettant d'envoyer un cours non masqué.

---

3. Authentification

JWT HS256, 7 jours, signé avec `JWT_SECRET`. Mots de passe par bcrypt, avec sel automatique ; l'entrée est tronquée à 72 octets explicitement, parce que bcrypt le ferait en silence.

Le message d'échec de connexion est identique que l'adresse soit inconnue ou le mot de passe faux — les distinguer publierait la liste des comptes.

4. Autorisation

Un seul type d'utilisateur, donc aucune garde par rôle. La seule règle d'accès est la propriété, vérifiée côté serveur à chaque requête via `cours_de(session, course_id, owner_id)`.

`Protege` dans le front ne protège **que l'affichage**. Le supprimer ne donnerait accès à aucune donnée. Deux tests vérifient qu'un étudiant reçoit 403 sur le cours d'un autre.

5. Secrets

Règle	Mise en œuvre
Jamais en dur	tout passe par `.env`
Jamais versionné	`.env` et `.env.*` dans `.gitignore`, sauf `.env.example`
Jamais exposé au client	la clé du modèle ne quitte pas le serveur
Jamais journalisé	`/api/health` ne rend que la **présence** de la clé
Rotation	une clé exposée est révoquée, pas réutilisée

Clé compromise connue : le jeton Hugging Face du fichier de correction de l'atelier 4 doit être considéré comme exposé. Ne jamais le réutiliser.

6. Variables d'environnement

Variable	Sensible	Rôle
`DATABASE_URL`, `DIRECT_URL`	**oui**	contiennent le mot de passe de la base
`JWT_SECRET`	**oui**	signature des jetons
`GEMINI_API_KEY` / `ANTHROPIC_API_KEY`	**oui**	renseigner seulement celle du fournisseur actif
`CLIENT_ORIGIN`, `PORT`, `LLM_*`, `SEGMENT_*`	non	configuration
`VITE_API_URL`	non	**publique** : tout ce qui est préfixé `VITE_` part dans le navigateur

7. Sécurité de l'API

- Toute entrée validée par Pydantic avant d'atteindre le code métier.
- CORS restreint à `CLIENT_ORIGIN`.
- Import PDF : type vérifié, 8 Mo maximum, extraction locale sans appel réseau.
- Les erreurs n'exposent jamais de trace, de requête SQL ni de chemin.
- **Pas de limitation de débit par utilisateur** : `RATE_LIMITED` existe dans la taxonomie mais n'est émis par aucune route. Limite assumée.

8. Injection de prompt

Le texte d'un cours peut contenir des consignes adressées au modèle — par accident ou non. Trois mesures :

1. le prompt du tuteur déclare explicitement que le cours est une **donnée** à analyser, jamais une instruction, et demande de signaler toute consigne rencontrée ;
2. les passages sont encadrés par des bornes `--- debut du passage n/m ---` et `--- fin du passage n/m ---` ;
3. la sortie est contrainte par un schéma : même détourné, le modèle ne peut pas renvoyer autre chose qu'un objet conforme, sous peine de rejet.

La troisième est la plus solide : elle ne repose pas sur l'obéissance du modèle.

9. Liste de contrôle

- [x] Authentification
- [x] Autorisation par propriété, vérifiée côté serveur
- [x] Validation de toutes les entrées
- [x] Validation de toutes les **sorties du modèle**
- [x] Secrets hors du dépôt
- [x] Anonymisation avant tout envoi, garantie par la structure

- [x] Les erreurs n'exposent rien d'interne
- [x] Import de fichier contrôlé
- [x] Défense contre l'injection de prompt
- [ ] Limitation de débit — **non fait**, assumé
- [ ] Journal d'audit — **non fait**, hors périmètre